8.2.3 거버넌스 시스템 커스터마이징

개요

이 고급 튜토리얼에서는 Substrate의 거버넌스 시스템을 커스터마이징하여 DAO(탈중앙화 자율 조직) 기능을 구현합니다. 투표 메커니즘, 제안 시스템, 실행 지연, 권한 관리 등을 포함한 완전한 거버넌스 프레임워크를 구축합니다.

학습 목표

- 거버넌스 팔렛 구조 이해 및 커스터마이징
- 다양한 투표 메커니즘 구현 (단순 다수결, 가중 투표, 위임 투표)
- 제안 생성, 검토, 실행 프로세스 구현
- 시간 기반 실행 지연 및 취소 메커니즘
- 역할 기반 권한 시스템 구현

사전 요구사항

- 고급 팔렛 개발 경험
- 거버넌스 개념 이해
- 암호학적 투표 시스템 기본 지식
- 시간 기반 로직 구현 경험

1단계: 커스텀 거버넌스 팔렛 구조

1.1 기본 데이터 구조

```
// pallets/custom-governance/src/lib.rs

#[derive(Encode, Decode, Clone, PartialEq, Eq, RuntimeDebug, TypeInfo)]

pub struct Proposal<AccountId, Balance, BlockNumber> {

pub proposer: AccountId,

pub title: Vec<u8>,

pub description: Vec<u8>,

pub call: Vec<u8>, // Encoded call

pub deposit: Balance,

pub created_at: BlockNumber,

pub voting_period: BlockNumber,

pub execution_delay: BlockNumber,

}


#[derive(Encode, Decode, Clone, PartialEq, Eq, RuntimeDebug, TypeInfo)]

pub enum VoteType {

Simple, // 단순 찬반

Weighted, // 가중 투표

Ranked, // 순위 투표

}


#[derive(Encode, Decode, Clone, PartialEq, Eq, RuntimeDebug, TypeInfo)]
```

```rust
pub struct Vote<Balance> {
pub vote_type: VoteType,
pub power: Balance,
pub conviction: u8, // 확신도 (잠금 기간과 연관)
pub delegated_to: Option<AccountId>,
}

#[derive(Encode, Decode, Clone, PartialEq, Eq, RuntimeDebug, TypeInfo)]
pub enum ProposalStatus {
Active,
Passed,
Rejected,
Executed,
Cancelled,
TimedOut,
}
```

1.2 핵심 팔렛 구현

```rust
#[frame_support::pallet]
pub mod pallet {
use super::*;
```

```rust
#[pallet::config]
pub trait Config: frame_system::Config {
type RuntimeEvent: From<Event<Self>> + IsType<<Self as frame_system::Config>::
type RuntimeCall: Parameter + Dispatchable<RuntimeOrigin = Self::RuntimeOrigin>
type Balance: Parameter + Copy + Ord + Zero + Saturating;
/// 최소 제안 보증금
#[pallet::constant]
type MinProposalDeposit: Get<Self::Balance>;
/// 기본 투표 기간
#[pallet::constant]
type DefaultVotingPeriod: Get<Self::BlockNumber>;
/// 기본 실행 지연
#[pallet::constant]
type DefaultExecutionDelay: Get<Self::BlockNumber>;
/// 최대 활성 제안 수
#[pallet::constant]
type MaxActiveProposals: Get<u32>;
}

#[pallet::pallet]
pub struct Pallet<T>(_);

/// 제안 저장소
```

```
#[pallet::storage]
pub type Proposals<T: Config> = StorageMap<
_,
Blake2_128Concat,
u32, // Proposal ID
Proposal<T::AccountId, T::Balance, T::BlockNumber>,
OptionQuery,
>;


/// 제안 상태
#[pallet::storage]
pub type ProposalStatuses<T: Config> = StorageMap<
_,
Blake2_128Concat,
u32,
ProposalStatus,
ValueQuery,
>;


/// 투표 기록
#[pallet::storage]
pub type Votes<T: Config> = StorageDoubleMap<
_,
```

```
Blake2_128Concat,

u32, // Proposal ID

Blake2_128Concat,

T::AccountId,

Vote<T::Balance>,

OptionQuery,

>;


/// 투표 결과 집계

#[pallet::storage]

pub  type VotingResults<T: Config> = StorageMap<

_,

Blake2_128Concat,

u32,

(T::Balance, T::Balance), // (찬성, 반대)

ValueQuery,

>;


#[pallet::call]

impl<T: Config> Pallet<T> {

/// 새 제안 생성

#[pallet::call_index(0)]

#[pallet::weight(10_000)]
```

```
pub fn propose(
origin: OriginFor<T>,
title: Vec<u8>,
description: Vec<u8>,
call: Box<T::RuntimeCall>,
deposit: T::Balance,
) -> DispatchResult {
let proposer = ensure_signed(origin)?;
// 보증금 확인
ensure!(deposit >= T::MinProposalDeposit::get(), Error::<T>::InsufficientDeposit
// 제안 생성 로직...
Ok(())
}


/// 투표 실행
#[pallet::call_index(1)]
#[pallet::weight(10_000)]
pub fn vote(
origin: OriginFor<T>,
proposal_id: u32,
approve: bool,
vote_power: T::Balance,
conviction: u8,
```

```
) -> DispatchResult {
let  voter = ensure_signed(origin)?;
// 투표 로직 구현...
Ok(())
}


/// 제안 실행
#[pallet::call_index(2)]
#[pallet::weight(10_000)]
pub  fn  execute_proposal(
origin: OriginFor<T>,
proposal_id: u32,
) -> DispatchResult {
ensure_signed(origin)?;
// 실행 조건 확인 및 실행...
Ok(())
}
}
}
```

2단계: 고급 투표 메커니즘

2.1 위임 투표 시스템

```
impl<T: Config> Pallet<T> {
/// 투표권 위임
#[pallet::call_index(3)]
#[pallet::weight(10_000)]
pub fn delegate_vote(
origin: OriginFor<T>,
delegate: T::AccountId,
conviction: u8,
) -> DispatchResult {
let delegator = ensure_signed(origin)?;
// 위임 로직 구현
// - 순환 위임 방지
// - 위임 체인 추적
// - 투표력 계산
Ok(())
}

/// 가중 투표 계산
fn calculate_vote_weight(
voter: &T::AccountId,
base_power: T::Balance,
conviction: u8,
```

```
) -> T::Balance {
// 확신도에 따른 가중치 적용
let multiplier = match conviction {
0 => 1, // 잠금 없음
1 => 2, // 1 기간 잠금
2 => 4, // 2 기간 잠금
3 => 8, // 4 기간 잠금
4 => 16, // 8 기간 잠금
5 => 32, // 16 기간 잠금
6 => 64, // 32 기간 잠금
_ => 1,
};
base_power.saturating_mul(multiplier.into())
}
}
```

2.2 쿼드라틱 투표

```
impl<T: Config> Pallet<T> {
/// 쿼드라틱 투표 실행
#[pallet::call_index(4)]
#[pallet::weight(10_000)]
pub fn quadratic_vote(
```

```
origin: OriginFor<T>,

proposal_id: u32,

vote_credits: T::Balance,

approve: bool,

) -> DispatchResult {

let voter = ensure_signed(origin)?;

// 쿼드라틱 비용 계산: cost = votes²

let vote_power = Self::sqrt(vote_credits);

let cost = vote_power.saturating_mul(vote_power);

ensure!(cost <= vote_credits, Error::<T>::InsufficientVoteCredits);

// 투표 기록 및 크레딧 차감

Self::record_quadratic_vote(proposal_id, voter, vote_power, approve, cost)?;

Ok(())

}



/// 정수 제곱근 계산

fn sqrt(n: T::Balance) -> T::Balance {

if n.is_zero() {

return T::Balance::zero();

}

let mut x = n;

let mut y = (n + T::Balance::from(1u32)) / T::Balance::from(2u32);

while y < x {
```

```
x = y;

y = (x + n / x) / T::Balance::from(2u32);

}

x

}

}
```

3단계: 시간 기반 실행 시스템

3.1 지연 실행 메커니즘

```
/// 예약된 실행 정보

#[derive(Encode, Decode, Clone, PartialEq, Eq, RuntimeDebug, TypeInfo)]

pub struct ScheduledExecution<BlockNumber, Call> {

pub proposal_id: u32,

pub call: Call,

pub scheduled_at: BlockNumber,

pub can_cancel_until: BlockNumber,

}


#[pallet::storage]

pub type ScheduledExecutions<T: Config> = StorageMap<

_,
```

```
Blake2_128Concat,
T::BlockNumber,
Vec<ScheduledExecution<T::BlockNumber, T::RuntimeCall>>,
ValueQuery,
>;


impl<T: Config> Pallet<T> {
/// 제안 실행 예약
fn  schedule_execution(
proposal_id: u32,
call: T::RuntimeCall,
delay_blocks: T::BlockNumber,
) -> DispatchResult {
let  current_block = frame_system::Pallet::<T>::block_number();
let  execution_block = current_block.saturating_add(delay_blocks);
let  cancel_deadline = execution_block.saturating_sub(T::MinCancelPeriod::get())
let  scheduled = ScheduledExecution {
proposal_id,
call,
scheduled_at: execution_block,
can_cancel_until: cancel_deadline,
};
ScheduledExecutions::<T>::mutate(&execution_block, |executions| {
```

```
executions.push(scheduled);
});
Ok(())
}


/// 블록 종료 시 예약된 실행 처리
fn on_finalize(block_number: T::BlockNumber) {
let executions = ScheduledExecutions::<T>::take(&block_number);
for execution in executions {
let result = execution.call.dispatch(frame_system::RawOrigin::Root.into());
match result {
Ok(_) => {
ProposalStatuses::<T>::insert(&execution.proposal_id, ProposalStatus::Executed);
Self::deposit_event(Event::ProposalExecuted {
proposal_id: execution.proposal_id,
});
},
Err(error) => {
Self::deposit_event(Event::ExecutionFailed {
proposal_id: execution.proposal_id,
error: format!("{:?}", error).into_bytes(),
});
}
```

```
}
}
}
}
```

4단계: 프론트엔드 통합

4.1 거버넌스 대시보드

```
// frontend/components/GovernanceDashboard.jsx
import React, { useState, useEffect } from 'react';

const GovernanceDashboard = ({ api, account }) => {
const [proposals, setProposals] = useState([]);
const [userVotes, setUserVotes] = useState({});
const [votingPower, setVotingPower] = useState('0');

const loadProposals = async () => {
if (!api) return;
try {
const proposalEntries = await api.query.customGovernance.proposals.entries()
const proposalData = await Promise.all(
proposalEntries.map(async ([key, proposal]) => {
```

```
const proposalId = key.args[0].toNumber();
const status = await api.query.customGovernance.proposalStatuses(proposalId)
const results = await api.query.customGovernance.votingResults(proposalId);
return {
id: proposalId,
...proposal.toJSON(),
status: status.toString(),
results: results.toJSON(),
};
})
);
setProposals(proposalData);
} catch (error) {
console.error('Error loading proposals:', error);
}
};


const submitVote = async (proposalId, approve, conviction) => {
if (!api || !account) return;
try {
const tx = api.tx.customGovernance.vote(
proposalId,
approve,
```

```
votingPower,
conviction
);
await tx.signAndSend(account);
alert('Vote submitted successfully!');
loadProposals();  // 새로고침
}  catch (error) {
console.error('Vote submission failed:',  error);
alert('Vote failed: '  +  error.message);
}
};


return (
<div  className="governance-dashboard">
<h2>Governance Dashboard</h2>
<div  className="proposals-list">
{proposals.map(proposal  => (
<ProposalCard
key={proposal.id}
proposal={proposal}
onVote={submitVote}
userAccount={account}
/>
```

```
))}
</div>
</div>
);
};

const ProposalCard = ({ proposal, onVote, userAccount }) => {
const [voteChoice, setVoteChoice] = useState(null);
const [conviction, setConviction] = useState(0);

return (
<div className="proposal-card">
<h3>{proposal.title}</h3>
<p>{proposal.description}</p>
<div className="proposal-stats">
<div>Status: {proposal.status}</div>
<div>Yes: {proposal.results[0]}</div>
<div>No: {proposal.results[1]}</div>
</div>
{proposal.status === 'Active' && (
<div className="voting-section">
<div className="vote-buttons">
<button
```

```
onClick={() => setVoteChoice(true)}
className={voteChoice === true ? 'selected' : ''}
>
Vote Yes
</button>
<button
onClick={() => setVoteChoice(false)}
className={voteChoice === false ? 'selected' : ''}
>
Vote No
</button>
</div>
<div className="conviction-slider">
<label>Conviction (Lock Period): {conviction}</label>
<input
type="range"
min="0"
max="6"
value={conviction}
onChange={(e) => setConviction(parseInt(e.target.value))}
/>
</div>
<button
```

```
onClick={() => onVote(proposal.id, voteChoice, conviction)}
disabled={voteChoice === null}
className="submit-vote"
>
Submit Vote
</button>
</div>
)}
</div>
);
};

export default GovernanceDashboard;
```

5단계: 실제 테스트 시나리오

5.1 거버넌스 테스트 스크립트

```
// tests/governance-integration.js
const { ApiPromise, WsProvider, Keyring } = require('@polkadot/api');

async function testGovernanceFlow() {
const wsProvider = new WsProvider('ws://localhost:9944');
```

```
const api = await ApiPromise.create({ provider: wsProvider });
const keyring = new Keyring({ type: 'sr25519' });
const alice = keyring.addFromUri('//Alice');
const bob = keyring.addFromUri('//Bob');
const charlie = keyring.addFromUri('//Charlie');

console.log('Testing complete governance flow...');

// 1. 제안 생성
console.log('1. Creating proposal...');
const proposalCall = api.tx.system.remark('Test governance proposal');
const proposeTx = api.tx.customGovernance.propose(
'Test Proposal',
'This is a test proposal for governance',
proposalCall,
1000000
);
await proposeTx.signAndSend(alice);
console.log('Proposal created');

// 2. 투표 실행
console.log('2. Submitting votes...');
// Alice 찬성 투표 (높은 확신도)
```

```
await  api.tx.customGovernance.vote(0, true, 1000000, 5)
.signAndSend(alice);
// Bob 반대 투표 (낮은 확신도)
await  api.tx.customGovernance.vote(0, false, 500000, 1)
.signAndSend(bob);
// Charlie 찬성 투표 (중간 확신도)
await  api.tx.customGovernance.vote(0, true, 750000, 3)
.signAndSend(charlie);
console.log('Votes submitted');


// 3. 투표 결과 확인
console.log('3. Checking vote results...');
const  results  =  await  api.query.customGovernance.votingResults(0);
console.log('Vote results:', results.toJSON());


// 4. 제안 실행 (투표 기간 종료 후)
console.log('4. Executing proposal...');
await  api.tx.customGovernance.executeProposal(0)
.signAndSend(alice);
console.log('Governance flow test completed');
}
```

```
testGovernanceFlow().catch(console.error);
```

요약

이 고급 튜토리얼에서는 다음을 구현했습니다:

1. **커스텀 거버넌스 시스템**: 제안, 투표, 실행의 완전한 워크플로우
2. **고급 투표 메커니즘**: 위임 투표, 쿼드라틱 투표, 확신도 기반 가중치
3. **시간 기반 실행**: 지연 실행, 취소 메커니즘, 자동 스케줄링
4. **사용자 인터페이스**: React 기반 거버넌스 대시보드
5. **통합 테스트**: 전체 거버넌스 플로우 검증

다음 튜토리얼에서는 브릿지 연결 구현을 다루겠습니다.

참고 자료

- Substrate 거버넌스 팔렛
- Polkadot 거버넌스 시스템
- OpenGov 구현